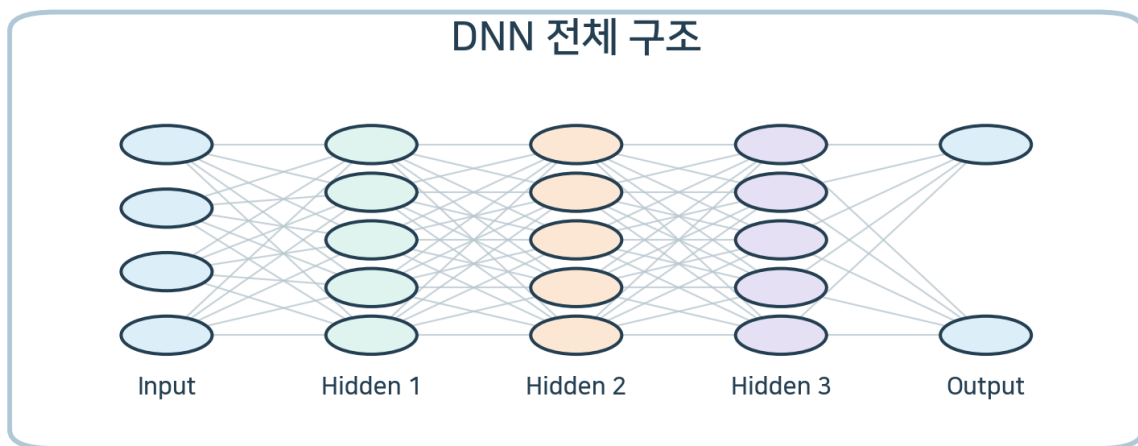


Deep Neural Network 기초

층, 가중치, 활성화 함수, 손실 함수, 학습 과정

이 문서는 데이터셋 개념을 이해한 뒤, 실제로 DNN이 입력 데이터를 어떻게 계산하고 학습하는지 설명한다. 단순히 용어를 외우는 것이 아니라, 데이터가 층을 통과하면서 어떻게 변형되고 왜 손실을 줄이면 예측 성능이 좋아지는지를 그림 중심으로 다룬다.



학습 목표

DNN의 전체 구조를 설명할 수 있다. 한 층의 계산 과정을 이해할 수 있다. Forward, Loss, Backpropagation, Update의 흐름을 연결해서 말할 수 있다. Activation function과 과적합 문제를 시각적으로 이해할 수 있다.

1. DNN이란 무엇인가?

DNN은 여러 개의 층을 가진 신경망이다. 입력 데이터는 첫 번째 층으로 들어가고, 여러 은닉층을 지나면서 계속 다른 표현으로 바뀐다. 마지막 출력층에서는 분류 결과나 예측값이 만들어진다.

핵심 정의

DNN = 입력 데이터를 여러 층으로 반복 변형하여 정답을 예측하는 함수 구조

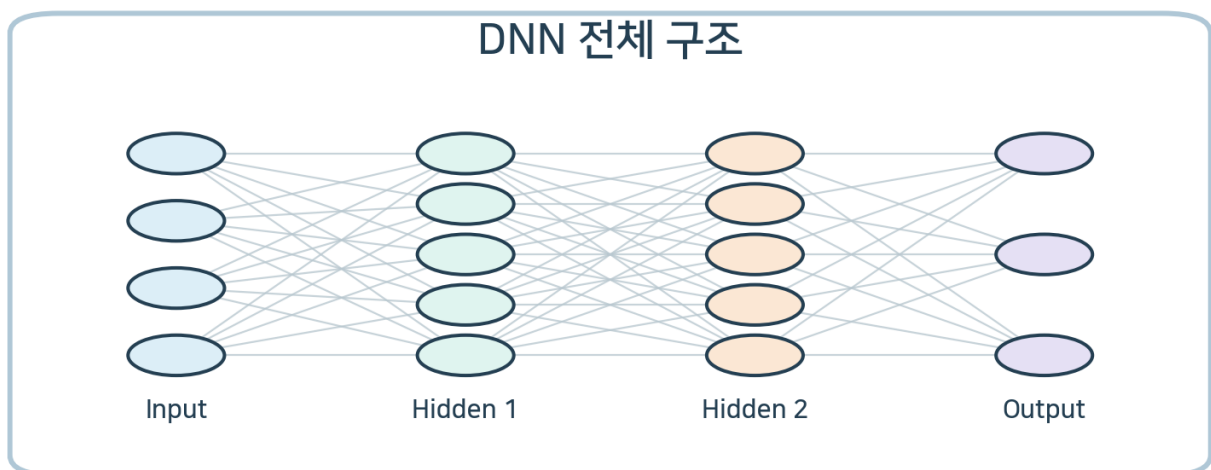


그림. 입력층, 여러 은닉층, 출력층이 연결되어 있으며 각 연결에는 가중치가 존재한다.

층이 깊어진다는 것은 단순히 선을 많이 연결한다는 뜻이 아니다. 각 층은 이전 층의 출력을 다시 계산하여 더 유용한 특징으로 바꾼다. 그래서 DNN은 원본 데이터에서 바로 정답을 찾기보다, 정답을 찾기 쉬운 중간 표현을 스스로 만들어간다.

2. 한 층에서 일어나는 계산

DNN의 각 층은 크게 두 단계를 수행한다. 먼저 입력값에 가중치를 곱하고 bias를 더하는 선형 계산을 한다. 그 다음 activation function을 적용해 비선형 변환을 만든다.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \rightarrow a^{(l)} = \sigma(z^{(l)})$$

여기서 W 는 가중치, b 는 bias, σ 는 activation function이다. 이전 층의 출력이 현재 층의 입력으로 들어오고, 현재 층의 출력은 다음 층으로 전달된다.

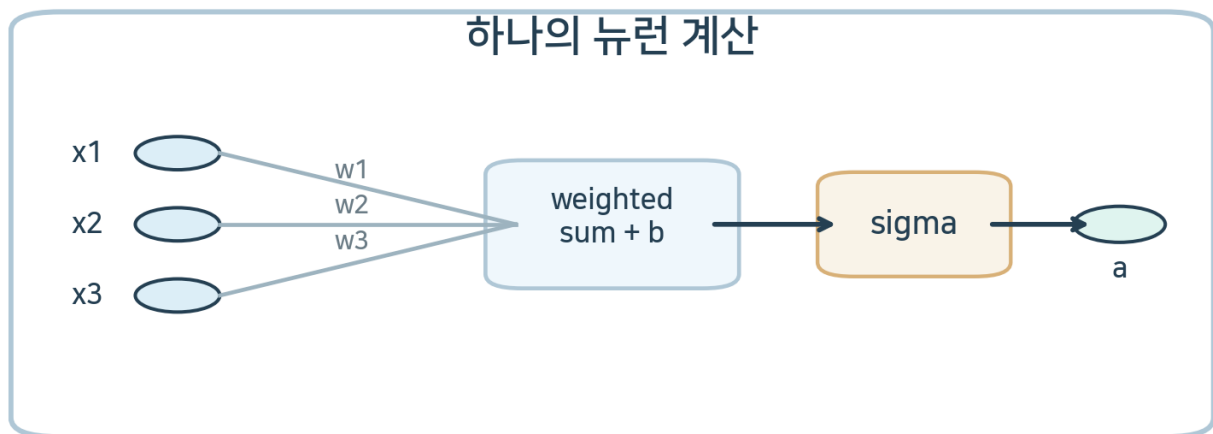


그림. 하나의 뉴런은 입력, 가중치, bias를 이용해 값을 만들고 activation function을 통과시킨다.

구성 요소	역할	직관
입력 a	이전 층에서 넘어온 값	모델이 보는 정보
가중치 W	입력의 중요도 조절	어떤 단서를 크게 볼지 결정
bias b	기준점 이동	판단 기준을 이동
activation	비선형 변환	복잡한 패턴을 표현

3. 왜 Activation Function이 필요한가?

여러 층을 쌓더라도 activation function이 없으면 전체 계산은 결국 하나의 선형 변환과 같다. 선형 변환만 반복하면 직선적인 규칙밖에 표현하지 못한다. DNN이 복잡한 패턴을 학습하려면 각 층 사이에 반드시 비선형성이 들어가야 한다.

구분	의미
선형만 있을 때	여러 층을 쌓아도 하나의 직선적인 변환으로 합쳐진다. 복잡한 경계를 만들기 어렵다.
비선형이 있을 때	층마다 공간을 꺾고 구부릴 수 있다. 복잡한 데이터 분포도 분리할 수 있다.

Activation이 표현력을 높이는 이유

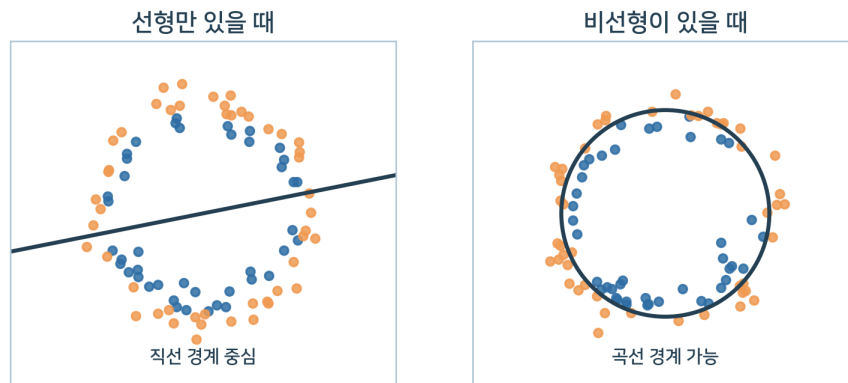


그림. 비선형성이 들어가면 직선만으로는 표현하기 어려운 경계도 만들 수 있다.

활성화 함수 그래프

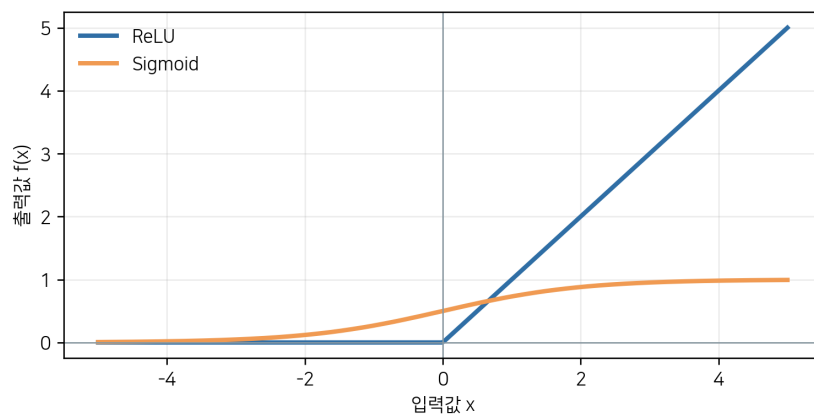


그림. ReLU와 Sigmoid는 입력값을 다른 형태로 바꾸어 모델의 표현력을 높인다.

4. 대표 Activation Function

Activation function은 모델의 표현력을 결정하는 핵심 요소이다. 은닉층에서는 ReLU 계열이 자주 사용되고, 확률 형태의 출력이 필요할 때는 sigmoid나 softmax 계열이 사용된다.

ReLU

$$f(x) = \max(0, x)$$

ReLU는 0보다 작은 값은 0으로 만들고, 0보다 큰 값은 그대로 통과시킨다. 계산이 단순하고 깊은 네트워크에서 gradient가 완전히 작아지는 문제를 줄이는 데 도움이 된다.

Sigmoid

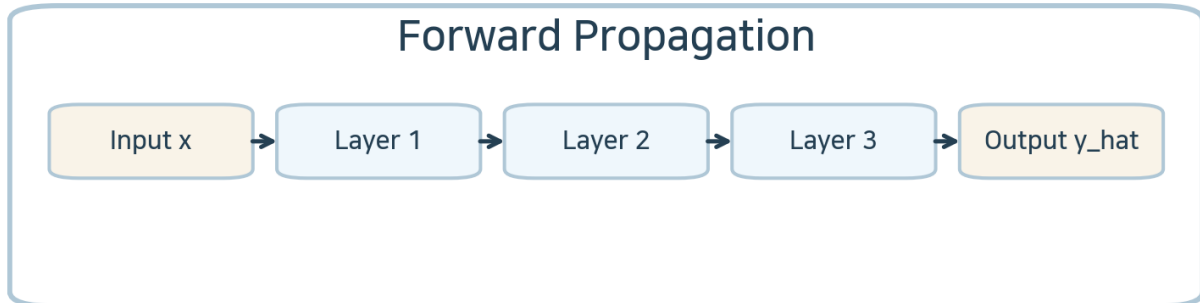
$$f(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid는 값을 0과 1 사이로 압축한다. 확률처럼 해석할 수 있지만, 입력값의 크기가 너무 커지거나 작아지면 기울기가 매우 작아져 깊은 네트워크에서는 학습이 느려질 수 있다.

함수	주요 특징	주의점
ReLU	계산이 빠르고 은닉층에서 많이 사용됨	음수 영역의 뉴런이 계속 0이 될 수 있음
Sigmoid	출력을 0과 1 사이로 압축함	깊은 층에서 gradient vanishing이 생길 수 있음

5. Forward Propagation: 예측값이 만들어지는 과정

Forward propagation은 입력 데이터가 모델을 통과하여 예측값이 만들어지는 과정이다. 입력은 첫 번째 층에서 계산되고, 그 결과가 다음 층의 입력이 된다. 이 과정이 출력층까지 반복된다.



$$a^{(0)} = x, \quad a^{(1)} = \sigma(W^{(1)}a^{(0)} + b^{(1)}), \quad \hat{y} = a^{(L)}$$

Forward는 현재 층의 출력이 다음 층의 입력이 되는 반복 구조이다. 마지막 층의 출력이 모델의 예측값이 된다.

단계	결과	의미
입력층	a0	원본 feature
은닉층 1	a1	첫 번째 변형 표현
은닉층 2	a2	더 추상화된 표현
출력층	y_hat	최종 예측

해석

Forward는 정답을 맞히기 위해 현재 가중치로 계산만 해보는 단계이다. 아직 가중치를 고치지는 않는다.

6. Loss Function: 얼마나 틀렸는지 측정하기

Loss function은 모델의 예측값과 실제 정답의 차이를 숫자로 나타낸다. 이 숫자가 작을수록 모델이 정답에 가까운 예측을 했다는 뜻이다.

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

회귀 문제에서는 예측값과 실제값의 거리 차이를 제공해서 손실로 사용할 수 있다.

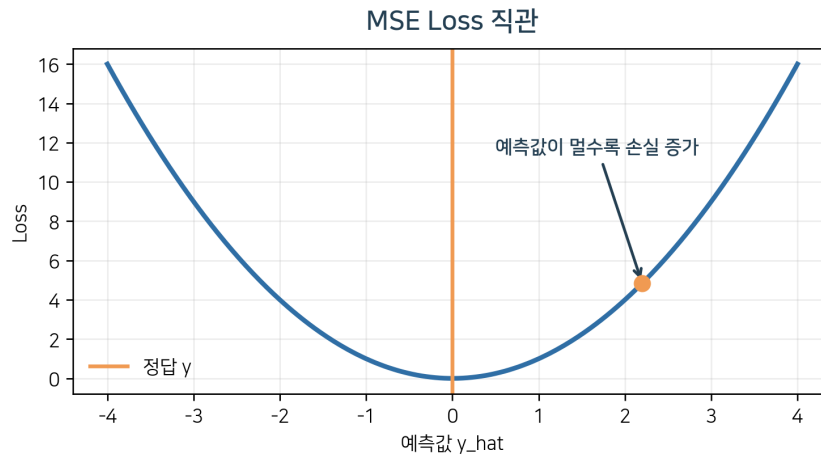


그림. 예측값이 정답에서 멀어질수록 손실은 커진다.

문제 유형	대표 손실 함수	직관
회귀	MSE	예측 숫자와 실제 숫자의 거리 최소화
다중 분류	Cross Entropy	정답 클래스에 높은 확률을 주도하도록 학습
이진 분류	Binary Cross Entropy	0 또는 1에 가까운 확률을 만들도록 학습

7. Cross Entropy: 분류에서 자주 쓰는 손실

분류 문제에서는 모델이 각 클래스에 확률을 준다. Cross Entropy는 정답 클래스에 높은 확률을 주지 못했을 때 손실을 크게 만든다. 그래서 분류 모델은 정답 클래스의 확률을 키우는 방향으로 학습된다.

$$L = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

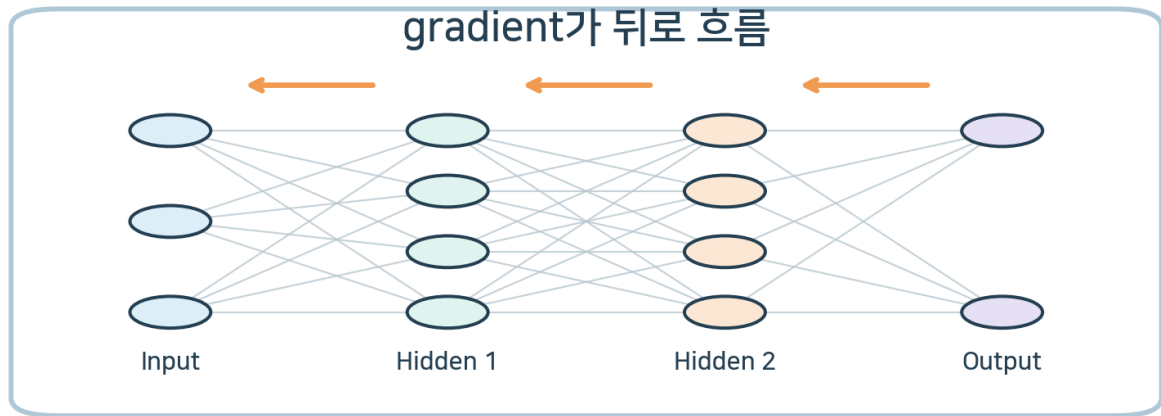
상황	정답 클래스 확률	손실 해석
좋은 예측	0.95	정답에 높은 확률을 줬으므로 손실이 작음
애매한 예측	0.50	확신이 부족하므로 손실이 중간 수준
나쁜 예측	0.05	정답에 거의 확률을 주지 않아 손실이 큼

중요한 해석

분류에서 loss가 작아진다는 것은 단순히 숫자 차이가 줄어드는 것이 아니라, 정답 클래스에 더 높은 확률을 주도록 모델이 바뀐다는 뜻이다.

8. Backpropagation: 오차를 뒤로 전달하기

Backpropagation은 loss를 줄이기 위해 각 가중치가 오차에 얼마나 영향을 주었는지 계산하는 과정이다. 출력층에서 시작한 오차 정보가 앞쪽 층으로 전달되기 때문에 역전파라고 부른다.



$$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial a^{(l)}} \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial W^{(l)}}$$

체인룰을 이용해 loss가 각 가중치에 미치는 영향을 계산한다. 이 값이 gradient이다.

직관

Backpropagation은 “어떤 가중치를 얼마나 고쳐야 loss가 줄어드는가?”를 찾는 과정이다.

9. Weight Update: 가중치를 실제로 수정하기

Backpropagation으로 gradient를 계산한 뒤에는 가중치를 업데이트한다. gradient는 loss가 증가하는 방향을 알려주기 때문에, 모델은 그 반대 방향으로 이동해야 한다.

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

eta는 learning rate이다. learning rate는 한 번에 얼마나 크게 이동할지를 정한다.

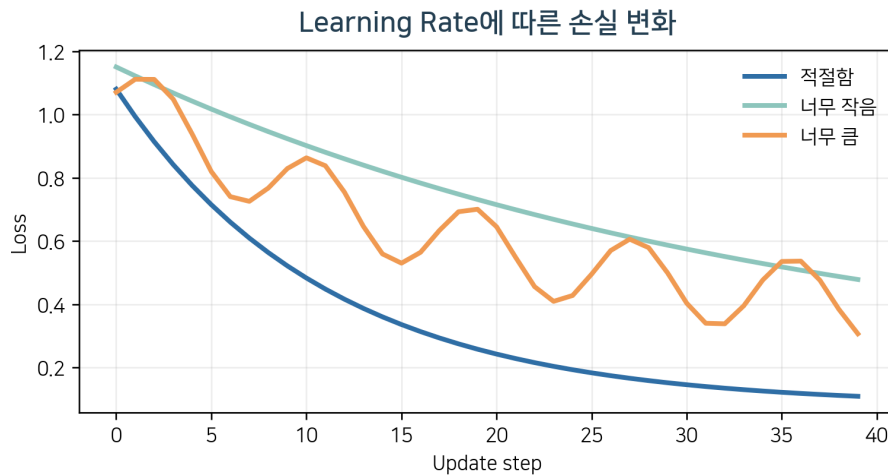


그림. Learning rate가 너무 작으면 느리고, 너무 크면 불안정해질 수 있다.

업데이트는 모델이 학습하는 실제 순간이다. Forward는 예측을 만들고, loss는 틀린 정도를 재고, backpropagation은 고칠 방향을 계산하며, update는 가중치를 바꾼다.

10. DNN이 하는 진짜 일: 표현 공간 변형

DNN의 핵심은 입력 데이터를 그대로 분류하는 것이 아니라, 층을 거치며 데이터를 분류하기 쉬운 표현 공간으로 바꾸는 것이다. 좋은 표현 공간에서는 복잡했던 문제가 더 단순한 경계로 해결될 수 있다.

DNN은 데이터를 분류하기 쉬운 공간으로 변형한다

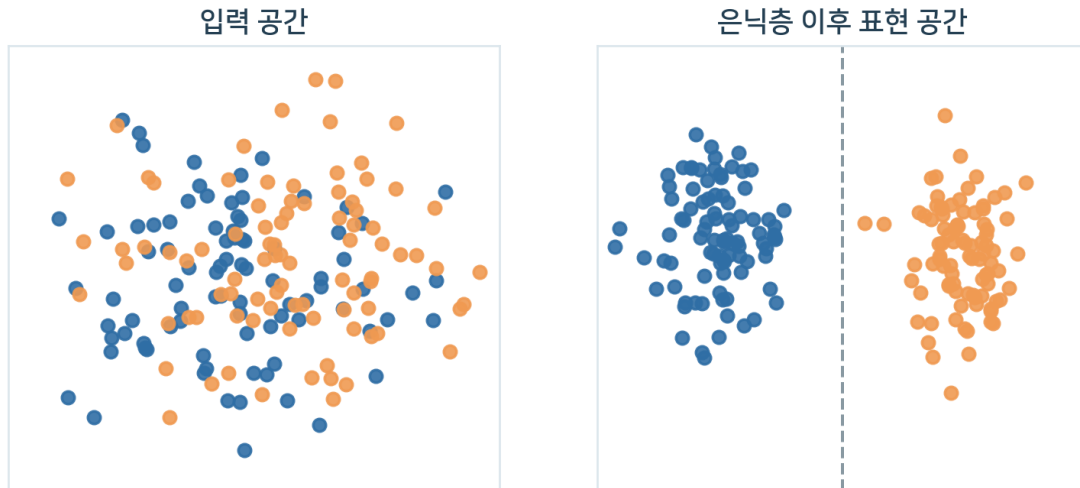


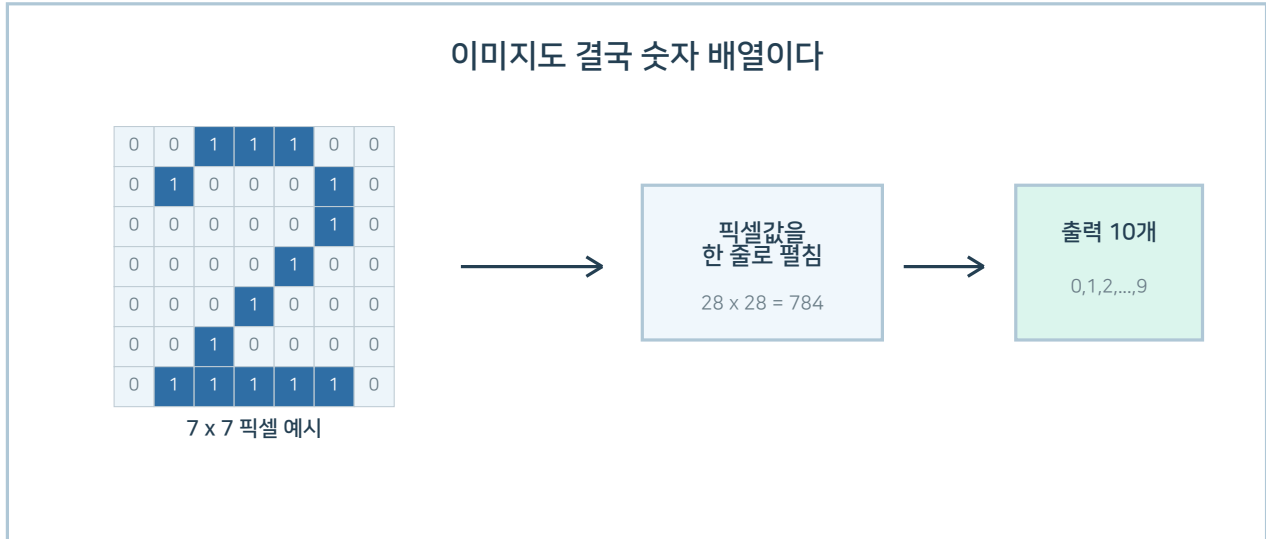
그림. 처음에는 두 클래스가 섞여 있어도, 은닉층 이후에는 더 잘 분리된 표현이 만들어질 수 있다.

중요한 해석

은닉층은 단순한 중간 계산이 아니다. 모델이 정답을 맞히기 위해 스스로 만든 새로운 feature라고 볼 수 있다.

11. 예시: 손글씨 숫자 분류

MNIST 같은 손글씨 숫자 분류에서는 이미지를 모델에 그대로 넣는 것이 아니라, 픽셀값을 숫자 벡터로 바꿔 입력한다. 28 x 28 이미지는 총 784개의 숫자로 표현된다.



구조 예시	의미
784	28 x 28 이미지의 픽셀값
128	첫 번째 은닉층의 뉴런 수
64	두 번째 은닉층의 뉴런 수
10	0부터 9까지 10개 클래스의 점수

12. DNN 학습 과정 전체 흐름

DNN 학습은 한 번의 계산으로 끝나지 않는다. 데이터를 여러 번 반복해서 보면서 예측, 손실 계산, 역전파, 업데이트를 계속 수행한다. 이 반복 단위를 보통 epoch라고 한다.

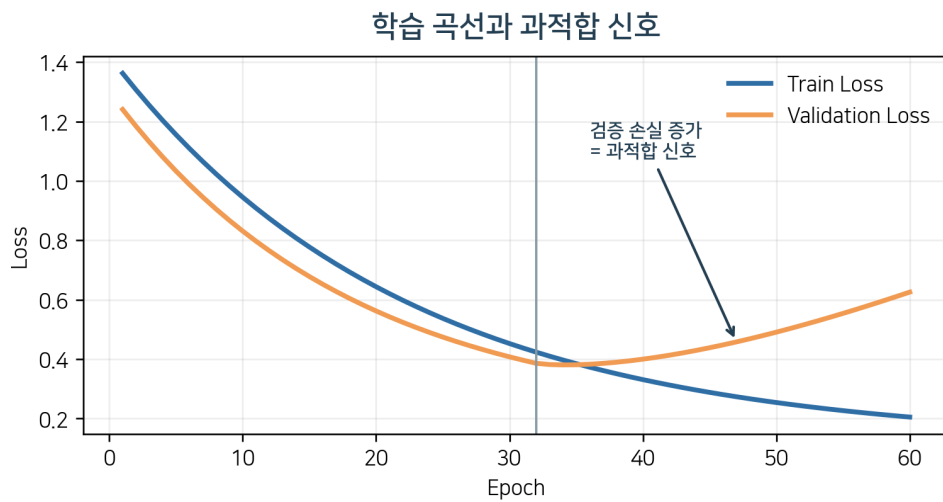
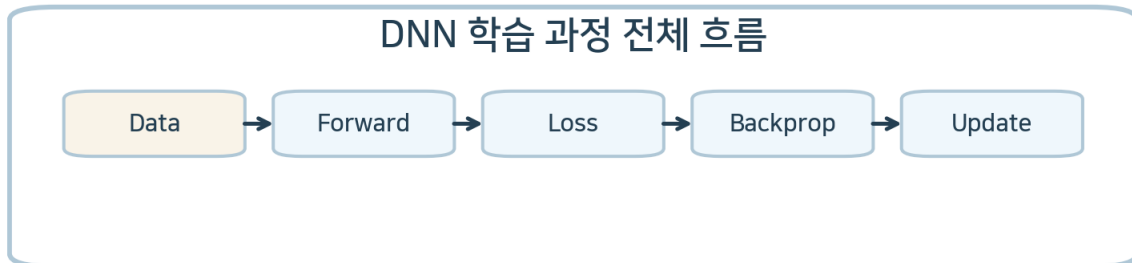
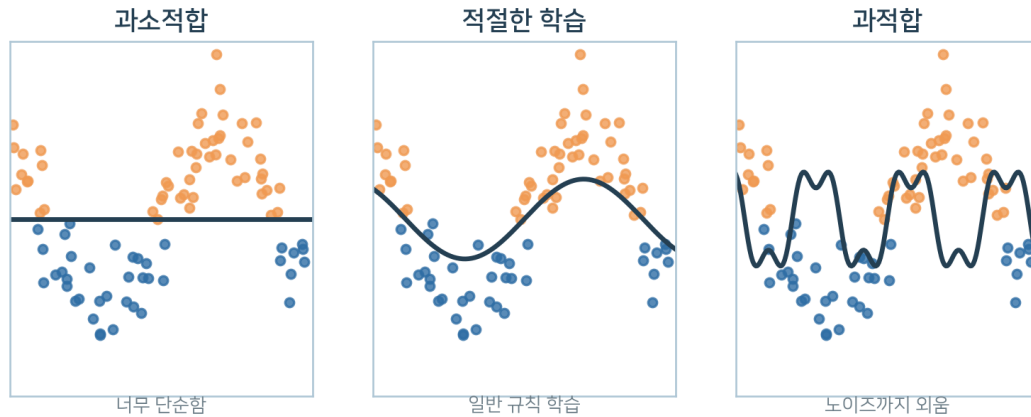


그림. 학습이 진행되면 train loss는 감소하지만, validation loss가 다시 증가하면 과적합 신호일 수 있다.

학습 곡선을 보는 이유는 모델이 실제로 좋아지고 있는지 확인하기 위해서다. train loss만 보면 학습 데이터에 잘 맞지만 알 수 있다. validation loss를 함께 봐야 새로운 데이터에 대한 성능을 추정할 수 있다.

13. 과적합: 학습 데이터를 외워버리는 문제

과적합은 모델이 학습 데이터에는 매우 잘 맞지만, 새로운 데이터에는 잘 맞지 않는 현상이다. DNN은 표현력이 강하기 때문에 데이터가 적거나 학습을 너무 오래 하면 작은 노이즈까지 외울 수 있다.



상태	Train 성능	Validation/Test 성능	해석
과소적합	낮음	낮음	규칙 자체를 잘 못 배움
정상 학습	좋음	좋음	일반화가 잘 됨
과적합	매우 좋음	낮음	학습 데이터를 외움

시험 비유

기출문제 답만 외운 학생은 같은 문제는 잘 맞지만 새로운 문제는 풀지 못한다. 과적합된 모델도 학습 데이터에는 강하지만 처음 보는 데이터에는 약하다.

14. 과적합을 줄이는 방법

과적합을 줄이는 핵심은 모델이 학습 데이터만 외우지 않고 새로운 데이터에도 통하는 규칙을 배우게 만드는 것이다. 이를 위해 데이터, 모델, 학습 과정을 함께 조절한다.

방법	핵심 아이디어	예시
데이터 추가	더 다양한 상황을 보여줌	다른 배경, 각도, 조명의 이미지 수집
Data Augmentation	기존 데이터를 변형해 다양성 증가	회전, 밝기 조절, 좌우 반전
Dropout	일부 뉴런을 임시로 끄م	특정 경로에만 의존하지 않게 함
Weight Decay	가중치가 지나치게 커지지 않게 함	복잡한 규칙 억제
Early Stopping	검증 성능이 나빠지기 전에 멈춤	validation loss 증가 시 중단

Dropout 직관: 일부 뉴런을 임시로 꺼서 경로 의존을 줄인다

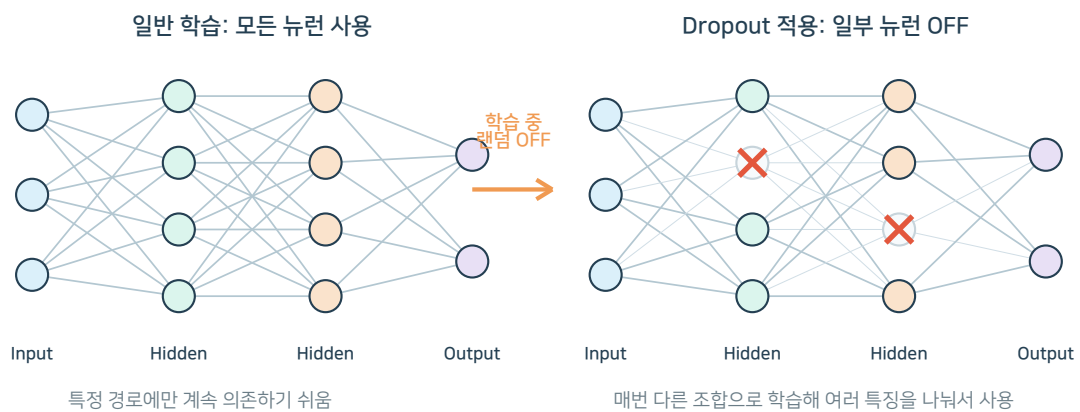


그림. Dropout은 학습 중 일부 뉴런을 끄면서 모델이 한 경로에만 의존하지 않게 만든다.

15. Batch, Epoch, Optimizer

DNN 학습에서는 데이터를 한 번에 전부 사용하지 않고 작은 묶음으로 나누어 학습하는 경우가 많다. 이 묶음을 batch라고 한다. 전체 데이터를 한 번 모두 학습에 사용하면 1 epoch가 끝난다.

용어	의미	예시
Batch	한 번 업데이트에 사용하는 데이터 묶음	32개 샘플씩 계산
Iteration	batch 하나로 한 번 업데이트	1000개 데이터, batch 100이면 10 iteration
Epoch	전체 데이터를 한 번 모두 사용	train set 전체를 한 바퀴 학습
Optimizer	gradient로 가중치를 바꾸는 알고리즘	SGD, Adam

Optimizer	직관
SGD	gradient 방향을 따라 단순하게 이동한다. 구조는 단순하지만 learning rate 선택에 민감할 수 있다.
Adam	각 파라미터마다 이동 크기를 조절한다. 많은 딥러닝 실습에서 기본 optimizer로 자주 사용된다.

해석

Batch size, learning rate, optimizer는 모델 구조 자체는 아니지만 학습 안정성과 속도에 큰 영향을 준다.

16. DNN을 볼 때 확인해야 할 체크리스트

DNN 구조를 설계하거나 학습 결과를 해석할 때는 아래 항목을 순서대로 확인하면 좋다. 성능이 낮다고 바로 층을 깊게 만드는 것보다, 데이터와 학습 흐름을 먼저 점검해야 한다.

점검 항목	확인 질문
입력 형태	feature의 차원과 범위가 올바른가?
출력 형태	분류 클래스 수와 출력 뉴런 수가 일치하는가?
손실 함수	문제 유형에 맞는 loss를 사용했는가?
활성화 함수	은닉층과 출력층의 activation이 적절한가?
학습 곡선	train loss와 validation loss를 함께 보고 있는가?
과적합 여부	train 성능만 높고 validation 성능은 낮지 않은가?
데이터 문제	라벨 오류, 클래스 불균형, 데이터 부족은 없는가?

실습 연결

블록 코딩 플랫폼에서는 사용자가 층 수, 뉴런 수, activation, loss, optimizer를 직접 바꾸게 된다. 따라서 문서에서는 각 설정이 학습 흐름에서 어떤 역할을 하는지 시각적으로 설명하는 것이 중요하다.

17. 전체 요약

- DNN은 여러 층을 가진 신경망이며, 입력 데이터를 반복적으로 변형해 예측값을 만든다.
- 한 층은 선형 계산과 activation function으로 구성된다.
- Forward propagation은 현재 가중치로 예측값을 계산하는 과정이다.
- Loss function은 예측값과 정답의 차이를 숫자로 나타낸다.
- Backpropagation은 loss를 줄이기 위해 각 가중치의 책임을 계산하는 과정이다.
- Update 단계에서 gradient의 반대 방향으로 가중치를 수정한다.
- DNN의 핵심은 데이터를 분류하기 쉬운 표현 공간으로 바꾸는 것이다.
- 과적합을 막으려면 데이터 품질, regularization, validation 성능을 함께 확인해야 한다.

한 줄 정리

DNN은 데이터를 여러 층으로 반복 변형하여, 정답을 예측하기 쉬운 표현으로 바꾸는 학습 구조이다.

다음 챕터 예고

다음 챕터에서는 DNN의 한계를 바탕으로 이미지 데이터에 특화된 CNN을 다룬다. 픽셀을 단순히 펼쳐서 보는 DNN과 달리, CNN은 공간 구조를 유지하면서 이미지의 지역 패턴을 학습한다.